

Q1 Formalising questions

25 Points

This question provides you a way to experiment with the steps that CoMAT follows.

Q1.1**5 Points**

Solve MQ1 and MQ2 using your calculation in any way you would like

MQ1:

r = number of rabbit pets

d = number of pet dogs

c = number of pet cats

t = total number of pets

$$d = 60$$

$$c = 2d = 2 \cdot 60 = 120$$

$$r = d + c - 12 = 60 + 120 - 12 = 168$$

$$t = d + c + r = 60 + 120 + 168 = 348$$

Final Answer: 348

MQ2:

Company X: Produced 5 chips in total, 1 is defective

Company Y: Produced 4 chips in total, 2 is defective

T : Total chips produced

$$T = 4 + 5 = 9$$

D : Total defective chips

$$D = 1 + 2 = 3$$

$P(D)$: Probability that the chip is defective = $\frac{D}{T} = \frac{3}{9} = \frac{1}{3}$

$P(Y \cap D)$: Probability that the chip is from company Y and defective = $\frac{2}{T} = \frac{2}{9}$

$P(Y|D)$: Probability that the chip came from Company Y, if the chosen chip is found to be defective

According to Bayes' Rule,

$$P(Y|D) = \frac{P(Y \cap D)}{P(D)} = \frac{\frac{2}{9}}{\frac{1}{3}} = \frac{2}{3}$$

Final Answer: d

Q1.2

5 Points

Follow the different steps.

MQ1, Identification and Definition

Variables: r (number of rabbit pets), c (number of pet cats), t (total number of pets)

Constants: D (number of pet dogs, 60)

MQ1, Structural Logic Translation

$$c = 2D$$

$$r = D + c - 12$$

$$t = D + c + r$$

$$r, c, t \geq 0 \text{ (non-negative constraints)}$$

MQ1, Explicit Factual Representation

$$D = 60$$

MQ1, Question Formalisation

Find the total number of pets t : $(c = 2D) \wedge (r = D + c - 12) \wedge (t = D + c + r)$

MQ2, Identification and Definition

Variables: D (total number of defective chips)

Constants: T_x (total chips from Company X, 5), D_x (defective chips from Company X, 1), T_y (total chips from Company Y, 4), D_y (defective chips from Company Y, 2), T (total number of chips, 9)

MQ2, Structural Logic Translation

$$D = D_x + D_y$$

$$P(D) = \frac{D}{T}$$

$$P(Y \cap D) = \frac{D_y}{T}$$

$$P(Y|D) = \frac{P(Y \cap D)}{P(D)}$$

MQ2, Explicit Factual Representation

$$T_x = 5$$

$$D_x = 1$$

$$T_y = 4$$

$$D_y = 2$$

$$T = 9$$

MQ2, Question Formalisation

Find the probability that the chip came from Company Y, given that the chosen chip is defective $P(Y|D) : (D = D_x + D_y) \wedge (P(D) = \frac{D}{T}) \wedge (P(Y \cap D) = \frac{D_y}{T}) \wedge (P(Y|D) = \frac{P(Y \cap D)}{P(D)})$. Choose an appropriate $P(Y|D)$ from following: a. $\frac{2}{9}$, b. $\frac{4}{9}$, c. $\frac{1}{2}$, d. $\frac{2}{3}$

Q1.3

5 Points

MQ1, Derivation of Final Answer

348

MQ2, Derivation of Final Answer

d

Q1.4

5 Points

Explain how the structured approach in 2-3 helped you arrive at an answer more mechanically. Did you feel it helped you reach a solution more easily? Note that there is no wrong or right answer here; you are expected to think critically about the process you followed.

I believe that the structured approach did not significantly enhance the problem solving process in MQ1 and MQ2. The formalisation steps felt excessive, as many parts could be solved intuitively without explicitly breaking them into steps.

For instance, in MQ1, I could easily determine the values of c and r while reading the question. It was a simple arithmetic problem and the formalisation was more time-consuming than problem-solving itself. Similarly, in MQ2, I could intuitively apply Bayes theorem without explicitly stating the formulas. Intuitively, I could sense that the answer would be $\frac{2}{3}$, as there were three defective chips in total and two of them came from Company Y.

However, I believe that this structured steps can be highly beneficial in more complex problems with multiple layers of mathematical reasonings. It enhances structure and clarity of the variables and logical relationships. This well-defined structure helps guide problem-solving process as it allows us to see what steps should be taken next, especially in problems that requires sequential reasoning. It also reduces potential mistakes, and if they occur, it becomes significantly easier to revisit the reasoning process, identify the source of the mistake, and make necessary corrections.

Q1.5

5 Points

Following subquestion 2, do you think further formalisation step(s) could be added? Can you give an example of such a step? You may also argue that no further formalisation steps are required and that the sequence is complete.

After working through the questions above, I believe an additional formalisation step can be added to enhance the reasoning of the model.

Unlike the example MQ0, as MQ2 was a multiple choice question. So if the steps were followed exactly as in MQ0, the model would generate an answer (i.e. 2/3) —although correct—does not directly map to the designated multiple-choice option (i.e. option (d)). This could lead to incorrect final answer, even if the mathematical reasoning was correct, especially if the model is not explicitly instructed to align its answer with one of the provided choices.

To address this issue, I have put an additional sentence "Choose an appropriate $P(Y|D)$ from following:" at step 4. However, I believe separating this as another step (that maps the result to the answer format) will logically be better, as each step will only serve one distinct purpose, which could enhance clarity and better reasoning traceability. Moreover, we could ensure that the problem was solved by mathematical reasoning alone, and not influenced by the answer choices.

However, this issue did not arise in MQ1, where no multiple-choice options were provided. This suggests that such a step should be optional, rather than enforced in all questions.

I believe one extra optional formalisation step should be sufficient. When compared to the human reasoning process demonstrated in Q1-1, the existing steps in CoMAT already align closely with how humans approach mathematical problems. Adding unnecessary steps could potentially hinder model performance by creating redundancy between steps, or breaking single reasoning units into overly granular substeps. Increasing the number of steps might complicate the prompt unnecessarily without enhancing meaningful gains in the quality of its logic. It is often found that prompt length and cognitive load can affect the quality of output in LLM.

Q2 Analysis of steps through Shapley values

35 Points

In this question, you will experiment with measuring the contributions of the different steps to the performance of CoMAT.

Disclaimer: We are aware that some of the code required for this assignment may be available online in some form. You are expected to write the code on your own – and there are tools to identify code similarity that indicate a suspected copying of code compared to a baseline code.

Q2.1

5 Points

Please upload your code here. The code should be uploaded as separate Python files.

CoMAT_instruction.py:


```
1  """
2  Question 3
3  Create a variable `INSTRUCTION` and fill in the prompt with instructions provided in
4  the file `prompt-instruction.txt.`
5  """
6  INSTRUCTION = """Task Description: You are given a college mathematics question.
7  Your task is to convert it into a symbolic representation and solve it using strict logical
8  reasoning, without any reliance on memorized answers or external knowledge.
9  Rigorously follow these steps:
10
11  1) Identify and define all the predicates, functions, and variables in the problem.
12  2) Parse the entire problem into logical rules based strictly on the defined predicates,
13  functions, and variables. Ensure that the multiple-choice options do not influence this
14  step in any way.
15  3) Write all the facts explicitly mentioned in the problem as logical statements.
16  4) Parse the question into a symbolic form using only the defined predicates and
17  variables, without any influence from the provided answer choices.
18  5) Solve the problem step by step using only the symbolic representations and logical
19  reasoning. Provide clear reasoning for each step.
20  6) Derive the final answer based solely on your symbolic solution and step-by-step
21  reasoning.
22  7) Only after deriving your answer, match it to the closest option provided, without
23  considering the order or labeling of the options.
24
25  IMPORTANT:
26  - Base your entire solution on the symbolic representation and logical reasoning.
27  - Do not rely on any prior knowledge, memorized answers, or previous examples.
28  - The order and labeling of options should not influence your reasoning or answer in
29  any way.
30  - Evaluate only the symbolic rules and facts you've derived, not the provided answer
31  choices.
32
33  After completing the symbolic representation and reasoning, provide the final answer
34  as a single letter corresponding to the option that best matches your derived answer.
35
36  This approach ensures an unbiased solution focused entirely on logical reasoning
37  through symbolic rules, minimizing any impact from option swapping or prior
38  knowledge.
39
40  The model should evaluate only the symbolic rules and facts provided and not
41  the provided answer choices to ensure an unbiased solution. Provide the final answer
42  based purely on logical reasoning as a single letter.
43
44  -----
45  Here is an example:
46  Question:
47  If  $f(x) = x^2 + 3x + 2$  and  $g(x) = 2x + 1$ , find  $f(g(2))$ .
48  A: 3
49  B: 42
```

```

33 C: 5
34 D: 30
35 E: 500
36
37 Example Answer:
38 ###
39 1. Define predicates and functions:
40   f(x): function that takes x and returns  $x^2 + 3x + 2$ 
41   g(x): function that takes x and returns  $2x + 1$ 
42
43 2. Parse problem into logical rules:
44    $\forall x: f(x) = x^2 + 3x + 2$ 
45    $\forall x: g(x) = 2x + 1$ 
46
47 3. Facts:
48   No additional facts given.
49
50 4. Parse the question:
51   Find  $f(g(2))$ 
52
53 5. Solve step by step:
54 [Think step by step and answer the question]
55 1. First, we need to calculate  $g(2)$ :
56  $g(2) = 2(2) + 1 = 4 + 1 = 5$ 
57 2. Now we have  $f(g(2)) = f(5)$ 
58 3. Calculate  $f(5)$ :
59  $f(5) = 5^2 + 3(5) + 2$ 
60  $= 25 + 15 + 2$ 
61  $= 42$ 
62
63 6. Derived answer: 42
64
65 7. Match to provided options: B
66
67 Final Answer: B
68
69 Note: The reasoning process was based solely on the symbolic representation and
70 logical rules, without relying on the order or labeling of the options.
#raise NotImplementedError("Create a variable `INSTRUCTION` and fill in the prompt
with instructions provided in the file `prompt-instruction.txt.`")

```

utils.py:

```
1 import io
2 import sys
3 import openai
4 import torch
5 from torch.nn import DataParallel
6 from dotenv import load_dotenv
7 import os
8
9 load_dotenv()
10
11 openai.api_key = os.getenv('OPENAI_API_KEY')
12 #####
13 ###Question 1: Create a .env file with the api keys with the input OPENAI_API_KEY=
   before executing the code###
14 #####
15 #raise NotImplementedError("Create a .env file with the api keys with the input
   OPENAI_API_KEY= before executing the code")
16 client = openai.OpenAI()
17
18 def predict_gpt(openai, messages):
19     #####
20     ###Question 2: INSERT THE CODE HERE###
21     response = openai.chat.completions.create(
22         model="gpt-4o-mini",
23         messages=messages,
24         temperature=0.0,
25         max_tokens=2000
26     )
27     return response.choices[0].message.content.strip()
28     #####
29     #raise NotImplementedError("Build the OpenAI function here based on the
   configurations given in the assignment")
30     """
31     This function, `predict_gpt`, is designed to interact with OpenAI's GPT model to
   generate predictions
32     based on a conversation history.
33
34     Args:
35         openai: The OpenAI client object used to make API calls.
36         messages: A list of dictionaries representing the conversation history,
37                   where each dictionary has a "role" (e.g., "system", "user", or "assistant")
38                   and "content" (the message text).
39         configurations: the model used should be gpt-4o-mini, with temperature of 0.
40         Note: Please do not change the gpt model, as higher accuracy does not lead to a
   higher mark in the assignment.
41         The assessment would mainly be assess the correctness of the implementation,
   rather than the performance
42
43     Returns:
```

```

44     The model's response as a string.
45     """
46
47
48     ## In the final step, you can use client.chat.completions.create(..., model="gpt-4o-
mini", ...) to create the model (complete)
49     ## As mentioned above, make sure you use *only* gpt-4o-mini with your OpenAI
key!
50
51     return
52
53 def model_evaluation(model_type, model, tokenizer, system_content, question,
formatted_options, device=None):
54     if model_type == "gpt":
55         messages = [
56             {"role": "system", "content": system_content},
57             {"role": "user", "content": f"Question:
{question}\n\nOptions:\n{formatted_options}"}
58         ]
59         model_result = predict_gpt(model, messages)
60     else:
61         raise ValueError(f"Unknown model_type: {model_type}")
62
63     print(f"Model result: {model_result}")
64     return model_result
65

```

shapley_value_evaluation.py:

```
1 import pandas as pd
2 import itertools
3 import numpy as np
4
5 """ In the following docstrings, the values are given just to exemplify the format of
6 the output, not to give the actual "real" output
7
8 steps = [1, 2, 3, 4]
9
10 def get_missing_steps(row, steps):
11     """
12     Get missing steps as a tuple for each row.
13
14     Example:
15     Input: row = {'step1_missing': 1, 'step2_missing': 0, 'step3_missing': 1,
16 'step4_missing': 0},
17     steps = [1, 2, 3, 4]
18     Output: (2, 4)
19     """
20     missing_steps = []
21
22     #####
23     """Question 4: INSERT CODE HERE: Get missing steps as a tuple for each row."""
24     for step in steps:
25         if row[f'step{step}_missing'] == 0:
26             missing_steps.append(step)
27
28     #####
29     #raise NotImplementedError("Get missing steps as a tuple for each row.")
30     return tuple(sorted(missing_steps))
31
32 def generate_all_subsets(steps):
33     """
34     Generate all possible subsets for the missing steps.
35
36     Example:
37     Input: steps = [1, 2, 3]
38     Output: [(), (1,), (2,), (3,), (1, 2), (1, 3), (2, 3), (1, 2, 3)]
39     """
40     all_subsets_missing = []
41     for r in range(len(steps) + 1):
42         subsets_r = list(itertools.combinations(steps, r))
43         all_subsets_missing.extend(subsets_r)
44     return all_subsets_missing
45
46 def compute_v_S(df, all_subsets_missing):
47     """
48     Compute v(S) for all subsets of missing steps.
```

```

46 Example:
47 Input:
48 df = pd.DataFrame({'missing_steps': [(), (1,), (2,), (1, 2)], 'is_correct': [0.8, 0.7, 0.6,
0.5]})
49 all_subsets_missing = [(), (1,), (2,), (1, 2)]
50 Output:
51 v_S = {(): 0.8, (1,): 0.7, (2,): 0.6, (1, 2): 0.5}
52 """
53
#####
54 ###Question 5: INSERT CODE HERE: Compute v(S) for all subsets of missing
steps.###
55 v_S = {}
56 for subset in all_subsets_missing:
57     subset_rows = df[df['missing_steps'] == subset] # filter out rows of df, where
missing_step of df = subset value (i.e. all the rows with missing_step (1,2))
58     v_S[subset] = subset_rows['is_correct'].mean() # from all the datas that has same
missing_step subset, find an average of success for that subset
59
#####
60 #raise NotImplementedError("Implement the code to compute v(S) for all subsets of
missing steps.")
61 return v_S
62
63 def compute_marginal_contributions(steps, v_S):
64     """
65     Compute the marginal contributions for each step.
66
67     Example:
68     Input:
69     steps = [1, 2]
70     v_S = {(): 0.85, (1,): 0.67, (2,): 0.72, (1, 2): 0.75}
71     Output:
72     Delta_sum = {1: 0.08, 2: 0.12}, valid_permutations_count = 2
73     """
74     permutations = list(itertools.permutations(steps))
75     Delta_sum = {i: 0.0 for i in steps}
76     valid_permutations_count = 0
77
78     total_steps_set = set(steps)
79
80     for pi in permutations:
81         valid_permutation = True
82         for i in steps:
83             idx_i = pi.index(i)
84
85 #####
86     ###Question 6: INSERT CODE HERE: Retrieve S_i, S_i_union_i, S_i_sorted,
S_i_union_i_sorted###
87     S_i = set(pi[:idx_i]) # included set of setps
88     S_i_union_i = S_i.union({i})
89     S_i_sorted = tuple(sorted(S_i))

```

```

89     S_i_union_i_sorted = tuple(sorted(S_i_union_i))
90     missing_S_i_sorted = tuple(sorted(set(steps) - set(S_i))) # missing steps
91     missing_S_i_union_i_sorted = tuple(sorted(set(steps) - set(S_i_union_i)))
92
93     #####
94     #raise NotImplementedError("Implement the retrieval of S_i, S_i_union_i, and
their sorted tuples.")
95     v_S_i = v_S.get(missing_S_i_sorted, np.nan)
96     v_S_i_union_i = v_S.get(missing_S_i_union_i_sorted, np.nan)
97     if np.isnan(v_S_i) or np.isnan(v_S_i_union_i):
98         valid_permutation = False
99         break
100     else:
101
102     #####
103     ###Question 6.1: INSERT CODE HERE: Compute the marginal contribution of
step i###
104     Delta_i = v_S_i_union_i - v_S_i #delta_i( $\pi$ )
105     Delta_sum[i] += Delta_i #Calculates sum of delta_i( $\pi$ ) over all  $\Pi$  (So in
compute_shapley_values, only calculate division. Different from equation)
106
107     #####
108     #raise NotImplementedError("Implement the computation of the marginal
contribution of step i.")
109     if valid_permutation:
110         valid_permutations_count += 1
111     return Delta_sum, valid_permutations_count
112
113 def compute_shapley_values(Delta_sum, valid_permutations_count, steps):
114     """
115     Compute the Shapley values for each step.
116
117     Example:
118     Input:
119     Delta_sum = {1: 0.08, 2: 0.12}
120     valid_permutations_count = 2
121     steps = [1, 2]
122     Output:
123     Shapley_values = {1: 0.04, 2: 0.06}
124     """
125
126     #####
127     ###Question 7: INSERT CODE HERE: Compute the Shapley values###
128     shapley_values = {}
129     for i in steps:
130         shapley_values[i] = Delta_sum[i] / valid_permutations_count
131
132     #####
133     #raise NotImplementedError("INSERT CODE HERE: Compute the Shapley values")
134     return shapley_values
135
136 def main():

```

```

132 df = pd.read_csv('evaluation_with_steps.csv')
133
134 df['missing_steps'] = df.apply(get_missing_steps, axis=1, args=(steps,))
135
136 #####
137 # Question 7.1: INSERT CODE HERE: Generate all possible subsets for the missing
steps#
138 all_subsets_missing = generate_all_subsets(steps)
139
140 #####
141 # Question 7.2: INSERT CODE HERE: Compute v(S)#
142 v_S = compute_v_S(df, all_subsets_missing) # dictionary containing v_S for every
possible subset (pi)
143 #####
144
145 #####
146 # Question 7.3: INSERT CODE HERE: Compute the Shapley values#
147 Delta_sum, valid_permutations_count = compute_marginal_contributions(steps, v_S)
148 shapley_values = compute_shapley_values(Delta_sum, valid_permutations_count,
steps)
149 print(shapley_values)
150 #####
151 #raise NotImplementedError("Implement the code to generate all possible subsets
for the missing steps.")
152 #raise NotImplementedError("Implement the code to compute v(S).")
153 #raise NotImplementedError("Implement the code to compute the Shapley values.")
154 if __name__ == "__main__":
155     main()

```


Q2.2

15 Points

Here answer the questions about the evaluation of the code execution.

(a) Accuracy of the model with temperature 0.0:

75.76

(b) Accuracy of the model with temperature 0.7:

73.74

(c) Explanation about the accuracy with temperature 0.0 versus 0.7:

The accuracy of the model under temperature 0.0 was observed to be 75.76%, and it decreased to be 73.74 when the temperature was increased to 0.7, which was about difference of two wrong questions.

Temperature value controls the randomness of the token generated in the next step, by directly changing the softmax distribution. With lower temperature as 0.0, the model enforces more deterministic choices mostly based on the tokens with highest probabilities. In contrast, a higher temperature as 0.7 changes the probability distribution of the tokens to be more uniform than the previous setting. This means that the model with higher temperature configuration is more likely to select more tokens from the tail of the distribution, increasing both the diversity of generated outputs and possibility of selecting a risky output.

The reason for the decrease in the accuracy after increasing the temperature to 0.7 is due to the nature of the task and the definition of temperature. In the context of the mathematical problems, determinism can play a crucial role, as a slight error in the process, such as using a wrong symbol or using synonyms instead of strict mathematical terms, could alter the result in a great extent. At a lower temperature, the model is more likely to generate more 'safe' output in the head of the distribution and generate consistent output instead of a diverse one.

On the other hand, higher temperature has more possibility to choose less probable token as an output, so it has more possibility to generate linguistically or semantically similar alternatives, instead of generating the most likely and conventional mathematical notation.

Analysing from the comat_gpt_normal.json for temperature 0.7, a clear

example is the reasoning for the question about divisibility by 11, where the higher-temperature model produced verbose, repetitive algebra without clearly simplifying the modular arithmetic expression $3x + 7y \equiv 0 \pmod{11}$. Instead of systematically simplifying the expressions and deriving a logical conclusion, it repeatedly restated incorrect or irrelevant algebraic manipulations, ultimately failing to recognize the correct relationship between expressions. Thus, the higher-temperature model output was mathematically incorrect due to unnecessary complexity and lack of precise logical deduction.

(d) Explanation about alternative configurations:

Apart from the temperature setting, several other configurations could be altered to influence the performance of gpt-4o-mini model.

1. top_p

Top-p limits the probability p from which the model samples its next tokens during nucleus sampling. Similar to the temperature described above, a lower value in top_p samples from the head of the distribution, which leads to more deterministic output. In contrast, higher values may increase diversity but also risk introducing distractive or logically incoherent steps.

2. max_tokens

max_tokens is the maximum number of tokens generated by the model. If the value is set too low, the model could possibly truncate important steps in the CoMAT or cut out the final answer at the end, undermining the advantage of the CoMAT. On the other hand, if it is excessively high, it might lead to unnecessarily verbose outputs or encourage the model to over-explain trivial steps, which could also negatively impact the performance of the model.

3. presence_penalty

Positive value of presence penalty penalize new tokens based on their previous appearance in the text. Setting this as a positive value, which discourages the model from using repeating tokens, could negatively impact the performance in CoMAT. This is because mathematical reasoning often require repetition of same variables and expression and the strict mathematical notation and terminologies that are re-used throughout the problem-solving process.

4. frequency_penalty

Positive values of frequency penalty penalizes new tokens based on the frequency of their previous appearance. Again, in tasks like CoMAT where

repeated symbolic references are expected to appear, frequency penalty may suppress necessary repetition and decrease the performance of the model.

5. stop

stop sequences define where the model should terminate generation after detecting specific phrases or tokens. This ensures that the output remains structured and terminates after the essential part of the response is generated. Incorporating a well-defined stop sequence could potentially enhance performance, as the CoMAT method and the main.py evaluation pipeline are designed to parse outputs in a structured format. By ensuring a clean endpoint in generation, the model can avoid trailing content that may interfere with final answer extraction and evaluation.

Q2.3

15 Points

(a) Shapley values of the different steps.

Shapley values of step 1:

0.036253115107429124

Shapley values of step 2:

0.04065355515143354

Shapley values of step 3:

0.03240273006892528

Shapley values of step 4:

0.019201409936912068

(b) Which steps receives the highest Shapley value and why?

Step 2 received the highest Shapley value, with a value of approximately 0.04065, which indicates that this step contributes most significantly to model performance, hence mathematical reasoning of the CoMAT.

Step 2 is a Structural Logic Translation Step. In this stage, CoMAT translates the natural language problem into a set of formal rules that capture the relationships between key variables, typically in the form of algebraic equations or constraints.

This step is critical because the model is not merely rewriting the problem - it is actively selecting which mathematical formulas and constraints to apply, based on its internal mathematical knowledge and understanding of the problem structure. In other words, Step 2 is where the model decides which equations should be applied in the problem-solving process. This involves retrieving the appropriate reasoning schema, such as conservation equations, geometric relations, probability laws, and correctly mapping them to the problem given.

While Step 1 (Identification and Definition) introduces the necessary entities and constants, it does not by itself provide reasoning capability - it is Step 2

that gives structure to these elements by formally connecting them into a solvable mathematical framework. Without Step 2, subsequent steps - such as inserting numerical values (Step 3) or formalising the question (Step 4) - would be ineffective, as they would lack the mathematical relationships required for computation.

This interpretation aligns with the prompt design outlined in the provided prompt-instruction.txt file, where Step 2 explicitly guides the model to parse the problem into formal logical rules before any symbolic reasoning or answer derivation occurs. For instance, in the example prompt, equations $\forall x : f(x) = x^2 + 3x + 2$, and $\forall x : g(x) = 2x + 1$, which is the crucial mathematical concept required to solve the problem, only appears in step 2.

Moreover, the high Shapley value of Step 2 reflects its consistent importance across different permutations of steps. Shapley values are computed over all possible orderings and combinations of steps, meaning that Step 2's contribution is not confined to a specific position in the reasoning chain. Instead, it consistently enhances performance regardless of whether it appears early or late, and even when paired with only a subset of other steps. This robustness demonstrates that Step 2 fundamentally determines how the model interprets the problem mathematically, yielding high marginal contributions across permutations.

(c) Explanation about relationship between the steps and the order of the Shapley values:

Order of the Shapley Values:

1. Step 2: 0.04065
2. Step 1: 0.03625
3. Step 3: 0.03240
4. Step 4: 0.01920

This ordering of Shapley values shows an implicit hierarchy in the reasoning process of CoMAT, reflecting the extent to which different stages contribute to the overall model performance and the reasoning of CoMAT prompting. The Shapley value provides a measure of each step's marginal contribution, averaged across all possible permutations of steps. Thus, this ordering reflects each step's contribution also when it appears early, late, or in subset of a reasoning sequence.

1. Step 2 (Structural Logic Translation) received the highest Shapley value (0.04065), indicating that it plays a foundational role in the reasoning process. As discussed in the answer to the question above, it is responsible

for not only restating information in mathematical notation, but selecting and applying reasoning schemas such as equations, constraints, or functional relationships. This step formalises the structure of the problem and applies mathematical premises and equations before solving the problem. In effect, Step 2 is what allows the CoMAT pipeline to transition from understanding the problem to executing a solution.

Importantly, Shapley value reflects how well a step contributes both on its own and in combination with others. If we examine $V(S)$ values (success of a subset of these n steps), we can see that Step 2 alone achieves the highest standalone performance among all steps:

$V(S)$ When only step 4 is used: 0.775578

$V(S)$ When only step 3 is used: 0.798680

$V(S)$ When only step 2 is used: 0.811881

$V(S)$ When only step 1 is used: 0.782178

This demonstrates that Step 2 contributes meaningfully even in isolation — further justifying its dominant Shapley value. Moreover, because a specific which includes or excludes a single step appear more frequently in the permutation (as shown by the equation in Q4), their marginal contribution is also weighted more heavily in the Shapley computation, reinforcing the centrality of this step.

2. Step 1 (Identification and Definition) ranks next (0.03625), offering a strong but slightly lesser contribution. This step defines the relevant variables and constants. Well-defined symbolic components help the model interpret problem content more accurately.

$V(S)$ When only step 1 is missing: 0.782178

$V(S)$ When only step 2 is missing: 0.795380

$V(S)$ When only step 3 is missing: 0.805281

$V(S)$ When only step 4 is missing: 0.815182

Interestingly, while Step 1 alone yields lower performance than Step 3 ($V(S)$ when only one step is used, its real impact becomes evident when examining exclusion-based performance: when Step 1 is the only step missing, performance drops significantly, lower than every other steps, even including Step 2, is excluded.

This indicates that Step 1, though not effective in isolation, significantly amplifies the effectiveness of other steps by providing the necessary definitions that other steps depend on. Therefore, Step 1 is a strong

supportive contributor — indispensable in combination, even if not generative by itself.

3. Step 3 (Explicit Factual Representation) provides moderate contribution (0.03240). It integrates all relevant facts mentioned in the question into the logical structure, avoiding omission of key numerical information.

In addition to grounding symbolic expressions with constants, Step 3 helps maintain consistency in factual information - a crucial benefit given that LLMs often introduce small deviations or hallucinations in longer reasoning chains. However, Step 3 has a more limited role when used in isolation. Without structural logic to contextualise these values, they become floating parameters with no computational context. For instance, in prompt-instruction.txt, the example problem includes "No additional facts given," indicating that facts alone do not enable meaningful reasoning. In such cases, if only Step 3 is used, the model would gain little or no benefit from the CoMAT process.

Nonetheless, Step 3 performs better than Step 1 when used alone ($V(S) = 0.798680$), likely because factual details can occasionally provide partial reasoning cues. However, when Step 3 is excluded ($V(S) = 0.805281$), the performance drop is relatively small, indicating that much of its information can be recovered or inferred from other steps. This suggests that while Step 3 could contain important information, it is largely supplemental when structural elements are already present.

4. Step 4 (Question Formalisation) contributes the least, reflected in its significantly low Shapley value (0.01920). This result is due to the nature of the step itself. This step parses the question into a symbolic expression, such as identifying the target variable or formulating the explicit goal of the solution. However, the goal can be often implicitly inferred from the problem structure already formed through Steps 1–3.

The minimal importance of Step 4 is also evident on the individual values of $V(S)$. When only Step 4 is used, performance is the lowest among all single-step configurations ($V(S) = 0.775578$). Additionally, when Step 4 is excluded (i.e. when Steps 1, 2, and 3 are included), performance is highest across all single-step exclusions — $V(S) = 0.815182$, suggesting that Step 4's roles are often implicitly recoverable from other steps.

Therefore, these Shapley values for each steps reveals that successful mathematical reasoning in CoMAT depends most heavily on the steps that construct symbolic structure, not merely on those that represent factual

content or articulate goals. It highlights how CoMAT's design prioritises structured formalisation over superficial information extraction.

However, it is worth noting that all Shapley values remain positive, indicating that each step - even those with lower marginal gains - still contributes positively to model performance. The differences lie in how substantially each step adds value, rather than whether it helps at all. This nuanced interpretation reinforces that CoMAT is designed as a compositional pipeline, where structural steps are essential drivers of performance, while others play complementary roles.

(d) Explanation about the dependence between Step 1 and Step 2:

The empirical observation noted in Appendix D of the CoMAT paper reveals that removing both Step 1 and Step 2 results in a smaller performance drop than removing Step 1 alone - yielding 4.48% higher accuracy. On the surface, this seems counterintuitive, as removing more components of a structured reasoning pipeline should typically degrade performance further. However, this phenomenon is not anomalous but rather reflects intricate functional dependencies and partial redundancy across the steps in CoMAT.

Step 2 (Structural Logic Translation) is responsible for constructing mathematical equations based on the variable definitions provided in Step 1 (Identification and Definition). While Step 1 alone does not generate reasoning, it is essential for Step 2 to function correctly.

When Step 1 is removed, the definitions of key entities such as variables and constants are no longer available. As a result, Step 2 becomes ineffective: it may attempt to build equations, but the absence of these symbolic definitions renders Step 2's structural logic semantically ungrounded. As illustrated in the prompt example from the paper, variables and constants like v , r , T , P_v , P_r , and R are introduced in Step 1, and become operative when contextualized within equations like $v + r = T$ and $P_v \cdot v + P_r \cdot r = R$ in Step 2. Without Step 1, it is difficult for Step 2 to decide on which variable to use when forming equation, and could potentially lead incorrect representation of problem, such as using different variables for the same entity. It could also be difficult to interpret the variable during the problem-solving process, leading to an incorrect conclusion.

In contrast, when both Steps 1 and 2 are removed, the model avoids flawed symbolic structures and instead relies on Steps 3 (Explicit Factual Representation) and 4 (Question Formalisation). Though these offer less structural abstraction, they prevent semantic incoherence and maintain

reasoning consistency—leading to better performance than retaining Step 2 alone without its prerequisite.

This interpretation is further supported by the empirical data in Appendix D in the paper. As stated in the question, when Step 1 is removed, accuracy drops to 70.54% (and it increases even further to 77.29% when Step 4 is removed). However, when both Step 1 and Step 2 are removed, accuracy increases to 75.02%. Similarly, when Step 1 and 4 are removed, accuracy drops to 73.8%, but when Step 1, 2, 4 are removed, accuracy increases to 77.29%. However, in configurations where Step 1 and Step 3 are removed (76.23%), removing Step 2 as well causes a significant drop to 68.5%.

This data shows that exclusion of both Step 1 and 2 allows the model to rely on other steps, and results in a higher accuracy than configuration with exclusion of only Step 1. This data also implies that the model performs reasonably well when relying on Step 3 alone or with Step 4, but Step 4 alone is insufficient to maintain reasoning accuracy. It also suggests that even Step 2 - despite being ungrounded when Step 1 is absent - may be less harmful than having only Step 4.

This interaction effect is an important insight: some steps cannot function effectively without others, and including a structurally dependent step (Step 2) without its prerequisite (Step 1) may degrade performance more than excluding both entirely. This highlights that Step 1 and Step 2 do not contribute independently but jointly. Including one without the other may not yield optimal reasoning performance. Conversely, if both are removed, the model may default to simpler, less symbolic reasoning paths, which can still yield relatively high accuracy. In this case, Step 3 emerges as the primary compensatory mechanism. The improved performance in configurations that exclude both Step 1 and Step 2 confirms that reasoning coherence is better preserved when flawed symbolic logic is avoided altogether.

As noted in the paper: “removing both Steps 1 and 2 results in a smaller performance drop... suggesting overlapping functionalities where the model compensates using remaining steps.” This affirms that CoMAT’s steps are functionally interdependent, and effective reasoning arises not from isolated components but from their coordinated integration.

In conclusion, this result underscores that symbolic reasoning in CoMAT depends on the interplay between Step 1 and 2. The value of a step lies not only in its standalone contribution but in how well it interfaces with others. The case of Steps 1 and 2 exemplifies this collaborative structure, where cohesion - not accumulation - drives reasoning effectiveness.

Q3 Analysing a case in which CoMAT does not work

15 Points

Q3.1

5 Points

Write down the full set of steps CoMAT provides through the large language model for a question in which the final answer is incorrect.

Configuration:

- temperature = 0.0
- model = gpt-4o-mini
- max tokens = 2000

Question: Let x and y be positive integers such that $3x + 7y$ is divisible by 11. Which of the following must also be divisible by 11?

Option 0: $4x + 6y$

Option 1: $x + y + 5$

Option 2: $9x + 4y$

Option 3: $4x - 9y$

1. Define predicates and functions:

Let:

- $P(x, y)$: Predicate that states $3x + 7y \equiv 0 \pmod{11}$

2. Parse problem into logical rules:

- $P(x, y)$ is true if $3x + 7y \equiv 0 \pmod{11}$.

3. Facts:

- We know that $3x + 7y \equiv 0 \pmod{11}$.

4. Parse the question:

We need to determine which of the following expressions must also be divisible by 11:

- $A : 4x + 6y$

- $B : x + y + 5$

- $C : 9x + 4y$

- $D : 4x - 9y$

5. Solve step by step:

We will analyze each option to see if it can be derived from $3x + 7y \equiv 0 \pmod{11}$.

Step 1: Analyze Option A: $4x + 6y$

- We can express $4x + 6y$ in terms of $3x + 7y$:

$$4x + 6y = (3x + 7y) + (x - y)$$

- Since $3x + 7y \equiv 0 \pmod{11}$, we need to check if $x - y \equiv 0 \pmod{11}$ must hold. This is not guaranteed, so we cannot conclude that $4x + 6y \equiv 0 \pmod{11}$.

Step 2: Analyze Option B: $x + y + 5$

- We cannot derive $x + y + 5$ directly from $3x + 7y \equiv 0 \pmod{11}$. Thus, we cannot conclude that $x + y + 5 \equiv 0 \pmod{11}$.

Step 3: Analyze Option C: $9x + 4y$

- We can express $9x + 4y$ in terms of $3x + 7y$:

$$9x + 4y = 3(3x) + 4y = 3(3x + 7y) - 9y$$

- Since $3x + 7y \equiv 0 \pmod{11}$, we have:

$$9x + 4y \equiv 3(0) - 9y \equiv -9y \pmod{11}$$

- This does not guarantee that $9x + 4y \equiv 0 \pmod{11}$.

Step 4: Analyze Option D: $4x - 9y$

- We can express $4x - 9y$ in terms of $3x + 7y$: $4x - 9y = 4x + 7y - 16y = (3x + 7y) + (x - 16y)$

- Since $3x + 7y \equiv 0 \pmod{11}$, we need to check if $x - 16y \equiv 0 \pmod{11}$ must hold. This is not guaranteed, so we cannot conclude that $4x - 9y \equiv 0 \pmod{11}$.

6. Derived answer:

None of the options can be conclusively shown to be divisible by 11 based on the given condition $3x + 7y \equiv 0 \pmod{11}$.

7. Match to provided options:

Since none of the options must be divisible by 11 based on our analysis, we conclude that the answer is that none of the options are guaranteed to be divisible by 11.

Final Answer: None of the options must be divisible by 11.

Final Answer: -1

Correct Answer: 3

Q3.2
5 Points

Analyse the steps CoMAT follows and identify where it fails.

Step 1: In this step, CoMAT identifies the central predicate of the problem. It correctly captures the central predicate of the problem $P(x, y)$, a modular constraint between variables x and y . This correctly translates the natural language description to the mathematical symbols. However, variable x and y are not explicitly defined, showing that the model was not able to identify and define 'all' variables in the problem.

Step 2: The problem is logically translated into a modular arithmetic rule, with no ambiguity. $3x + 7y \equiv 0 \pmod{11}$ is correctly formalised.

Step 3: This step successfully extracts the fact explicitly mentioned in the problem - $3x + 7y \equiv 0 \pmod{11}$. Since this is the only provided constraint in the problem, there are no omissions in the question itself.

Step 4: CoMAT correctly reformulates the goal. It successfully convey that the essence of the question is finding out an expression that is also divisible by 11, and provides the expression as well. However, it is not strictly following the prompt, as it is not parsing the question into a symbolic form, but rather as a restatement of the natural language problem and incorporating the equations from the answer options.

Step 5: CoMAT attempts to solve the problem using only the symbolic representations and logical reasoning and provides clear reasoning for each step. The model also attempts to judge each expression's divisibility by 11 by loosely relating it to the original expression $3x + 7y$, without validating whether the remaining terms (i.e. $x - y$, $-9y$ $x - 16y$) are divisible by 11. This reasoning is incomplete because it overlooks alternative symbolic derivations that could validate equivalence under modulo 11.

For instance, when analyzing Option D ($4x - 9y$), the model attempts a surface-level decomposition without fully leveraging the predicate $3x + 7y \equiv 0 \pmod{11}$.

But, if we isolate y using the constraint:

$$7y \equiv -3x \Rightarrow y \equiv -3x \cdot 7^{-1} \pmod{11} \Rightarrow y \equiv -3x \cdot 8 \equiv -24x \equiv -2x \pmod{11}$$

Then,

$$4x - 9y \equiv 4x - 9(-2x) \equiv 4x + 18x \equiv 22x \equiv 0 \pmod{11}$$

which confirms Option D must be divisible by 11.

This kind of application of mathematical concept is completely missing from the model's execution. Therefore, Step 5 lacks fails to uncover the correct result.

Step 6: CoMAT logically derive the final answer based solely on the symbolic solution in Step 5. However, due to the failure to generate a correct answer in Step 5, the final answer in Step 6 is incorrect.

Step 7: Similar to Step 6, CoMAT validly matches the Step 6's answer, which is that none of the answers are correct. It was not affected by the order or labeling of the options. However, as the answer in Step 6 was incorrect, the matching of the options was incorrect as well.

Q3.3

5 Points

Can you now suggest a step that would help fix this failure? Try it out with an LLM. Did it indeed fix the wrong answer?

In Q3-2, we observed that although CoMAT correctly identified the modular constrained $3x + 7y \equiv 0 \pmod{11}$, it failed to use this to derive expressions like $x \equiv 5y \pmod{11}$ or $y \equiv -2x \pmod{11}$, which are necessary to evaluate the divisibility of operations such as $4x - 9y$, which resulted in the failure to reach the correct answer.

<Step>

To solve this issue, a step to derive alternative equivalent symbolic expressions using advanced algebraic reasoning was proposed. This new step, named "2.5. Symbolic Equivalence Derivation (MANDATORY)", is added immediately after the logical parsing stage (Step 2) in CoMAT's reasoning process.

The goal of this step is to uncover hidden equivalences or structural relationships, such as modular transformations, variable isolation, or contradiction-based deductions, that are not directly stated but are crucial for deeper reasoning in subsequent steps in more advanced mathematical problem.

This step ensures that important implications like $x \equiv 5y \pmod{11}$ from $3x + 7y \equiv 0 \pmod{11}$ are explicitly derived before attempting symbolic evaluation of expressions in Step 5.

<Implementation>

To do this, two parts were added to the existing CoMAT_Instruction.py

In task instruction (Between (2) and (3)),

"2.5) After parsing logical rules, derive alternative equivalent symbolic forms using advanced algebraic reasoning. This includes using multiple symbolic techniques variable isolation, modular transformation, and contradiction checking. The goal is to uncover deeper mathematical relationships that may assist reasoning in subsequent steps. This step is mandatory before proceeding to Step 3."

In the example (Between 2. and 3.),

"2.5. Symbolic Equivalence Derivation (MANDATORY):

- Derive equivalent symbolic expressions using algebraic reasoning.

In this case, we re-express $f(g(x))$ by symbolic substitution:

$$\begin{aligned}
 f(g(x)) &= (2x + 1)^2 + 3(2x + 1) + 2 \\
 &= 4x^2 + 4x + 1 + 6x + 3 + 2 \\
 &= 4x^2 + 10x + 6
 \end{aligned}$$

→ Equivalent expression derived: $f(g(x)) = 4x^2 + 10x + 6$

<Test with LLM>

To test the effect of the proposed step, it was tested in the exact same configuration as Question 3-1, with a change in CoMAT_instruction.py only.

The result, in summary, was successful. In Step 2.5, the model correctly derived the key mathematical equivalence $x \equiv 5y \pmod{11}$. This equivalence was then effectively utilized in Step 5 to evaluate the expressions, leading to the correct reasoning process. In Step 6, the model successfully identified the correct result, and in Step 7, it accurately matched the derived solution to the correct answer choice: D.

The full response generated by the model was following:

1. Define predicates and functions:

Let:

- $P(x, y)$: Predicate that states $3x + 7y \equiv 0 \pmod{11}$

2. Parse problem into logical rules:

From the problem, we have:

- $P(x, y)$ is true if $3x + 7y \equiv 0 \pmod{11}$

2.5. Symbolic Equivalence Derivation (MANDATORY):

We need to explore the implications of $3x + 7y \equiv 0 \pmod{11}$ to derive equivalent expressions.

1. Rearranging $3x + 7y \equiv 0 \pmod{11}$:

$$3x \equiv -7y \pmod{11}$$

This can also be expressed as:

$$3x \equiv 4y \pmod{11} \quad (\text{since } -7 \equiv 4 \pmod{11})$$

2. We can multiply both sides of the equivalence by the modular inverse of 3 modulo 11. The inverse of 3 modulo 11 is 4 (since $3 \cdot 4 \equiv 1 \pmod{11}$): $x \equiv 4 \cdot 4y \pmod{11} \implies x \equiv 16y \pmod{11} \implies x \equiv 5y \pmod{11}$ (since $16 \pmod{11} = 5$)

3. Facts:

$$- 3x + 7y \equiv 0 \pmod{11}$$

$$- x \equiv 5y \pmod{11}$$

4. Parse the question:

We need to check which of the following expressions must also be divisible by 11:

- $4x + 6y$
- $x + y + 5$
- $9x + 4y$
- $4x - 9y$

5. Solve step by step:

We will evaluate each expression under the condition $x \equiv 5y \pmod{11}$.

1. For $4x + 6y$:

$$4x + 6y \equiv 4(5y) + 6y \equiv 20y + 6y \equiv 26y \equiv 4y \pmod{11}$$

This is not necessarily divisible by 11.

2. For $x + y + 5$:

$$x + y + 5 \equiv 5y + y + 5 \equiv 6y + 5 \pmod{11}$$

This is not necessarily divisible by 11.

3. For $9x + 4y$:

$$9x + 4y \equiv 9(5y) + 4y \equiv 45y + 4y \equiv 49y \equiv 5y \pmod{11}$$

This is not necessarily divisible by 11.

4. For $4x - 9y$:

$$4x - 9y \equiv 4(5y) - 9y \equiv 20y - 9y \equiv 11y \equiv 0 \pmod{11}$$

This expression is divisible by 11.

6. Derived answer:

The expression $4x - 9y$ must be divisible by 11.

7. Match to provided options:

The closest option that matches our derived answer is:

D.

Final Answer: D

<Analysis of the Test Result>

In Q3-1, the failure stemmed from missing symbolic equivalence transformation: although CoMAT correctly identified the modular constraint $3x + 7y \equiv 0 \pmod{11}$, it failed to transform this into a usable form like $x \equiv 5y \pmod{11}$. Without this equivalence, Step 5 could not accurately evaluate the divisibility of expressions such as $4x - 9y$, and thus arrived at an incorrect or incomplete reasoning path.

With the new Step 2.5 in place, the model successfully derived the modular equivalence $x \equiv 5y \pmod{11}$ using modular inverse operations. This enabled a more precise substitution in Step 5:

- All expressions such as $4x - 9y$ could now be evaluated purely in terms of $y \pmod{11}$.
- As a result, the model correctly deduced $4x - 9y \equiv 0 \pmod{11}$, identifying the correct final answer (Option D).

This demonstrates that Step 2.5 effectively bridged the gap between structural constraints and practical symbolic evaluation, resolving the earlier failure mode and leading to correct and rigorous mathematical reasoning.

Additional Observation:

Since the example used to illustrate the task description of Step 2.5 included a modular transformation, it was important to assess whether the observed improvement was specific to modular arithmetic problems or extended more generally. To this end, the modified instruction was evaluated on the entire dataset, not limited to modular problems. The resulting accuracy was 76.77%, representing a slight improvement over the original model's performance (75.76%) under the same configuration (i.e. one additional correct prediction).

This result indicates that the addition of Step 2.5 may contribute to improved mathematical reasoning in general, though the extent of this enhancement remains relatively modest.

Q4 Using LLMs to prove an identity about the Shapley value

25 Points

Answer Q4 per instructions provided in the assignment.

Step 0: Equation (1), (2) and (3)

$$\Delta_i(\pi) = v(S_i \cup \{i\}) - v(S_i) \quad (1)$$

$$\phi_i = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \Delta_i(\pi) \quad (2)$$

$$\phi_i = \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot \frac{1}{n \cdot \binom{n-1}{|S|}} \quad (3)$$

Step 1. ϕ_i in Equation (1) and (2)

$$\Delta_i(\pi) = v(S_i \cup \{i\}) - v(S_i) \quad (1)$$

$$\phi_i = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \Delta_i(\pi) \quad (2)$$

Where:

Π is the set of permutations over the n players (steps),

$S_i \subseteq [n] \setminus \{i\}$ is the set of players (steps) that precede i in permutation π .

$$\phi_i = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} [v(S_i^\pi \cup \{i\}) - v(S_i^\pi)] \quad (1) (2)$$

as Π is set of permutations over the n players, $|\Pi| = {}^n P_n = n!$.

Therefore:

$$\phi_i = \frac{1}{n!} \sum_{\pi \in \Pi} [v(S_i^\pi \cup \{i\}) - v(S_i^\pi)] \quad (1) (2)$$

Step 2. Reduce S_i^π into S

Each subset $S \subseteq [n] \setminus \{i\}$ can repetitively appear as S_i^π in multiple permutation π .

Let z denote the frequency of subset $S \subseteq [n] \setminus \{i\}$ appearing as S_i^π (i.e. the set of players preceding i) in permutations $\pi \in \Pi$:

$$z = |S|! \cdot (n - 1 - |S|)!$$

Because:

In every permutation π , the set S_i^π is the set of steps that come before the step i

If the step i appears at the j -th position in a permutation ($j = 1, 2, 3, \dots, n$),

then $j - 1$ steps should come before it. So S_i^π must be a subset of size $j - 1$.

Therefore, a subset S of size $|S| = j - 1$ is created from a permutation in which i fixed at the j -th position. Hence, the number of such subsets can be directly counted via the number of corresponding permutations.

Note that for a given j , all subsets of size $j - 1$ are valid candidates for S_i^π , and only subsets of that size can occur—there is no need to sum over different values of j .

Consider a permutation π over n steps with i fixed at the position j :

- From position 1 to $j - 1$: subset S
- At position j : i
- From position $j + 1$ to n : permutation of all remaining elements $([n] \setminus \{\{i\} \cup S\})$

The number of such permutation can be calculated as follows:

- From position 1 to $j - 1$: $|S|!$ (There are $|S|!$ ways to permute elements in the subset S)
- At position j : 1 (fixed as one value, i)
- From position $j + 1$ to n : $(n - 1 - |S|)!$ (since the number of remaining elements is $n - (1 + |S|) = (n - 1 - |S|)$)
- Since these parts are independent given the fixed position of i , the total number of permutations for a fixed subset S is calculated by multiplying these values:

$$z = |S|! \cdot 1 \cdot (n - 1 - |S|)! = |S|! \cdot (n - 1 - |S|)!$$

This indicates that among the total $n!$ permutations, there are exactly $|S|! \cdot (n - 1 - |S|)!$ permutations in which $S_i^\pi = S$. This means that the same marginal contribution $\Delta_i(\pi)$ is repeated for each such permutation.

Therefore, the summation over permutations in Equation (2) can be transformed into a summation over subsets in Equation (3), weighted by the number of permutations where the subset appears.

$$\sum_{\pi \in \Pi} \Delta_i(\pi) = \sum_{\pi \in \Pi} v(S_i^\pi \cup \{i\}) - v(S_i^\pi) = \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot z$$

Step 3. Substitution of z into Equation (3)

Using the summation of Δ_i above, we can reduce the permutation by a set in Equation (1)(2) by subsets S .

$$\begin{aligned}
\phi_i &= \frac{1}{n!} \sum_{\pi \in \Pi} [v(S_i^\pi \cup \{i\}) - v(S_i^\pi)] \quad (1) \quad (2) \\
&= \frac{1}{n!} \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot z \\
&= \frac{1}{n \cdot (n-1)!} \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot |S|! \cdot (n-1-|S|)! \\
&= \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot \frac{|S|! \cdot (n-1-|S|)!}{n \cdot (n-1)!}
\end{aligned}$$

From the identity:

$$\binom{n-1}{|S|} = \frac{(n-1)!}{|S|! \cdot (n-1-|S|)!},$$

$$\phi_i = \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot \frac{1}{n \cdot \binom{n-1}{|S|}} \quad (3)$$

Conclusion: Therefore, Equation 2 and Equation 3 are proven to be equivalent.

<Use of LLM>

Initially, I developed the entire proof independently. My approach involved a combinatorial interpretation of Equation (3), leveraging expressions such as $\frac{n-1 P_{n-1}}{n-1 C_{|S|}}$ to approximate the frequency of subset appearances within permutations.

Use of LLM: ChatGPT Plus (GPT-4o)

Subsequently, I utilized ChatGPT Plus (GPT-4o) to explore alternative formulations and validate my reasoning. While the LLM was helpful in certain aspects, I encountered several limitations that are worth noting.

Difficulties of Using LLMs in Mathematical Proofs:

- Omission of Intermediate Reasoning Steps: A notable issue was the frequent omission of key logical steps, possibly due to the max_token constraints. For instance, the derivation of the subset frequency term $z = |S|!(n-1-|S|)!$ - which I considered central to the proof - was initially skipped. Only after a follow-up prompt did the model provide the appropriate combinatorial reasoning.

- Confusing Notational Substitution: The LLM often replaced general symbolic expressions with their derived numerical equivalents too early in the derivation. For instance, instead of using the abstract form such as $|\Pi|$ to denote the total number of permutations as typically done in formal definitions, it immediately substituted this with $n!$, even in the initial

formulation of the problem. Another example was its reformulation of Equation (3) as:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{n!}{|S|!(n-|S|-1)!} \cdot [v(S \cup \{i\}) - v(S)],$$

rather than the original form using binomial coefficients:

$$\phi_i = \sum_{S \subseteq [n] \setminus \{i\}} [v(S \cup \{i\}) - v(S)] \cdot \frac{1}{n \cdot \binom{n-1}{|S|}}.$$

This substitution was initially misleading, as chatGPT used this form directly from the beginning, and created confusion regarding whether the model had misunderstood the mathematical formulation.

- Symbolic Inconsistency and Silent Alteration: While the LLM's internal reasoning was self-consistent, it did not explicitly indicate changes it had made to variable names or symbols from the original equation given. (i.e. instead of $[n]$, it uses N). This created a confusion at first, and I had to carefully cross-reference notation and verify that logical equivalence was preserved across formulations when adding the result to the my answer.
- Equation Symbol Parsing: The model demonstrated a limited capacity to recognize mathematical notation when: 1) directly copied from PDF documents, 2) provided in image-based form, or 3) when the entire PDF was uploaded. Equation (1), (2), (3) was manually written to the chatGPT using Markdown notation.
- Rendering and Syntax Errors: The model occasionally omitted backslashes or misused LaTeX commands, resulting in incorrect equation rendering. Although these issues were minor, they were distracting and undermined clarity when transferring the content into formal write-ups.

Despite these limitations, I eventually found the LLM's alternative derivation of the z to be more concise and straightforward than my original formulation. Instead of my $\frac{n-1 P_{n-1}}{n-1 C_{|S|}}$ framework, the LLM's combinatorial derivation - based on fixing the position of player i and enumerating permutations of preceding and succeeding steps provided a clearer and more intuitive understanding. Consequently, I revised my entire Step 2. Reduce S_i^π into S structure to adopt this improved formulation.

<Further Exploration of LLMs>

In addition to the main analysis, several other LLMs were briefly explored for a comparison.

1. DeepSeek - DeepThink

DeepSeek followed a similar overall structure, deriving z from the permutation-based definition. The response began with a narrative-style explanation that mimicked human reasoning, guiding the reader through the logic using informal language and intuitive steps. This early part of the proof described how subset frequencies arise in permutations using concrete examples and descriptive reasoning. However, in the second part, Deepseek restated the same information in more formal algebraic steps. This transition made the proof both easy to follow and mathematically rigorous.

2. Gemini

Gemini's proof was shorter and more compact. It used similar core logic but skipped many of the intermediate explanations emphasized in other versions. While some approaches (including my derivation) focused on making each step conceptually clear, Gemini emphasized clarity in structure and simplicity rather than depth in reasoning.

3. chatGPT-mini 4o with CoMAT

(1) Q2 configuration: the code from Q2 was modified to apply the CoMAT framework for proving this equation. The model successfully completed the task, but not in depth. A substantial portion of the output was dedicated to outlining the CoMAT structure itself, rather than focusing on the core mathematical reasoning - likely due to the limited token length. The core derivation logic, such as z and the transition from Equation (2) to (3), were only briefly stated without deeper justification.

(2) max_token = 4000

Therefore, the number of tokens was increased to address the issue above. This version improved the derivation by explicitly including key reasoning steps - such as the calculation of z and how it connects to $\frac{1}{n \cdot \binom{n-1}{|S|}}$. While both versions shared the same structural clarity, the 4000-token version provided a more complete explanation of the mathematical logic

<Conclusion>

Overall, this work demonstrates that LLMs can perform symbolic reasoning remarkably well, offering effective assistance in formal derivations and logical structuring. However, their outputs are not immune to subtle errors or misinterpretations, and thus still require careful human verification to ensure full mathematical rigour and correctness throughout the reasoning process.